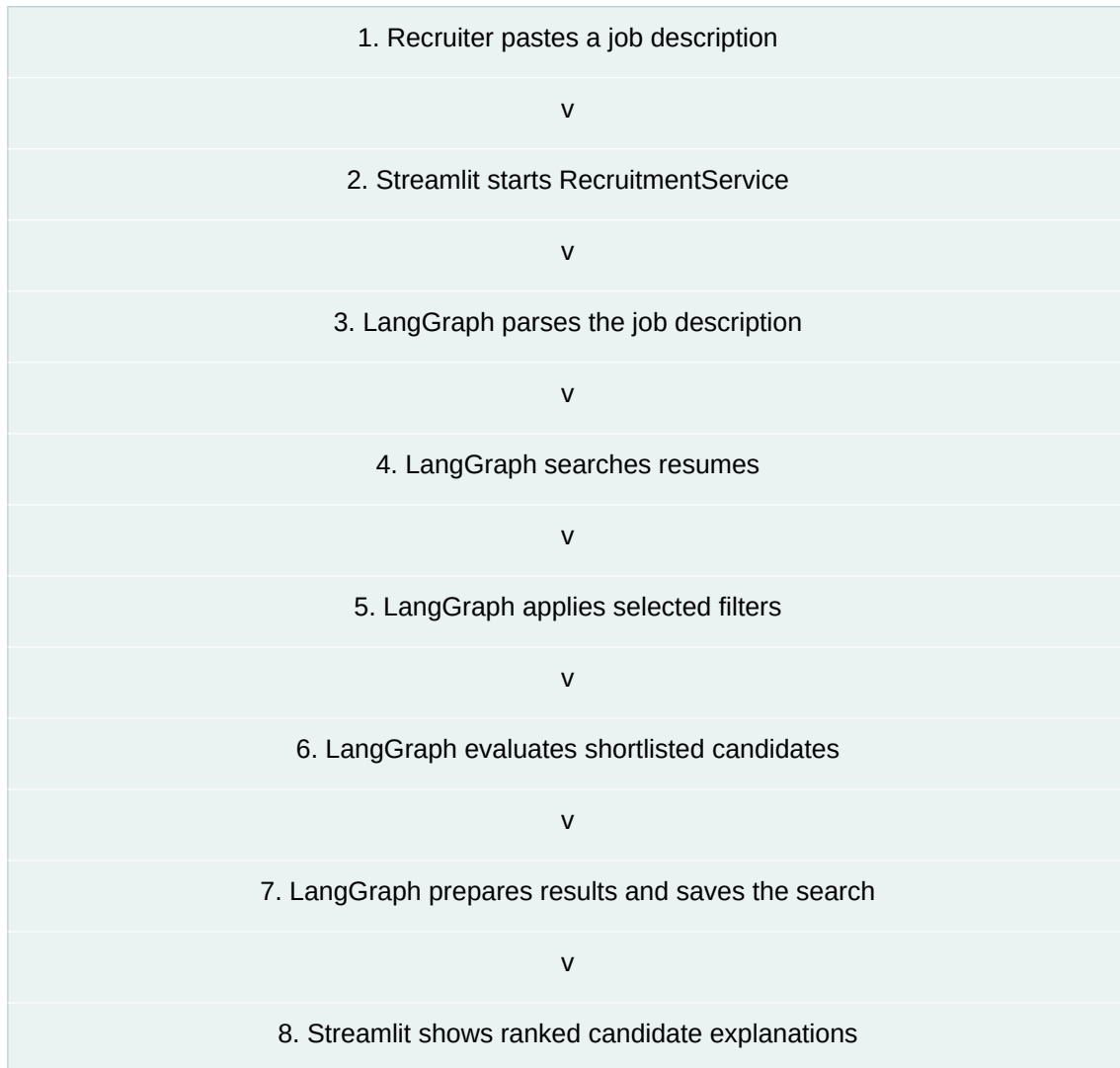


HireFlow Code Flow

A step-by-step guide to how a job description becomes a ranked candidate list.



The short version

HireFlow reads resume files saved on this computer. When a recruiter searches, the program looks for matching words, applies only the filters the recruiter selected, and explains each result using information found in the resume.

By default, search works without an internet connection or an API key. Gemini is optional and only used after the recruiter turns it on.

Step by Step

1. Start the application

Run **streamlit run streamlit/app.py**. The app creates the local data folders and shows the resume library and job-description form.

2. Add resumes

In the Resume library section, choose PDF, TXT, or Markdown files and click Add selected resumes.

core/ingestion.py saves them in **data/resumes/**. It changes unsafe filenames and avoids overwriting a file with the same name.

3. Read each resume

core/parsing.py extracts PDF or text content, cleans extra spaces, and looks for common skills, email, phone, location, and years of experience. **core/models.py** stores these details in a Candidate record. Long text is split into overlapping pieces so nearby details are not separated.

4. Read the job description

The first non-empty line is used as the job title. HireFlow looks for common skill names, a written experience requirement, and a line such as Location: Toronto. This is simple text matching, so extracted details should be checked.

Search and Results

5. Find resumes

core/hybrid_indexer.py compares words in the job description with words in each resume. Rare words matter more than very common words. The result is a search score between 0 and 1. If Gemini is enabled, the program can also compare text embeddings and combine the scores.

6. Apply filters

core/filters.py applies the selected must-have skills, location, and minimum experience. Leave a filter empty to keep candidates who do not meet that condition in the ranked list.

7. Score and explain

core/re_ranker.py compares candidate skills and experience with the job. It returns a score from 0 to 10, strengths, gaps, risks, a short summary, and exact lines from the resume as evidence. If Gemini is unavailable, these simple rules are used.

8. Show results and save activity

The Streamlit page shows the ranked candidates. If no results appear, it explains whether there are no resumes, no matches, or filters removed all candidates. Searches and review clicks are saved under **data/memory/**.

Files and Key Ideas

Where to look

streamlit/app.py - screen and button actions
core/recruitment.py - connects the steps in order
core/ingestion.py - reads and saves resumes
core/parsing.py - extracts text and simple fields
core/hybrid_indexer.py - finds and ranks likely resumes
core/filters.py - applies recruiter-selected filters
core/re_ranker.py - creates candidate explanations
core/models.py - defines Candidate, JobDescription, and CandidateEvaluation
core/memory_rag.py - stores search and review history

Fallbacks and limits

A damaged resume is skipped and reported; the other resumes are still searched. A scanned PDF needs OCR, which is not included. SQLite stores optional vectors locally; this starter project does not connect to Pinecone. Evaluation metrics are simple word-overlap estimates, not the full RAGAS framework.

Run checks

Run **python -m unittest discover -s tests -v** to run tests, or **python -m compileall -q .** to check Python syntax.